AI×ロボティクスによる創造的タスクの実現

SO-101 ピック＆プレース 第2版：2本指グリッパ × 摩擦把持

Physical AI 講座 2026 応用編1 最終課題 説明資料 ／ omnicampus アカウント名：maty0505

1. 課題の概要

タスク	テーブル上のランダムな位置にある赤いブロック（2.5 cm 角の剛体）を、SO-101 アームの2本指で挟み、**摩擦力だけ**で持ち上げて青いトレイに入れる
使用ツール	ROS 2 Jazzy ／ Gazebo Harmonic（物理エンジン DART）／ LeRobot 0.6.1 ／ PyTorch ／ Google Colab（T4 GPU）
AI の活用	模倣学習モデル **ACT** を学習し、その推論でロボットを自律制御（学習・推論の両方）。開発過程では生成 AI アシスタントをコード作成・デバッグの補助に使用
提出物	動画：学習済みポリシーによる自律実行（**等倍速**）／ コード一式：https://github.com/pai-claude-maty0505/second_model ／ 本資料

5 / 5
把持テスト（手本の動作）
摩擦だけで成功

60 / 65
手本デモの成功
（5,883 フレーム）

1.447 → 0.065
ACT 学習損失
（10,000 ステップ）

5 / 10
自律実行の成功
（最良の推論設定）

2. 目的

第1版では、指を閉じるとブロックを手先に「固定」する補助（DetachableJoint）を使っていました。また、指とブロックが衝突しない設定のため、見た目では指がブロックに食い込んでいました。第2版では実機に近い条件として、次の 3 点を目的にしました。

- ブロックを、指と衝突する**剛体**にする
- ハンドの**両側の指が開いて閉じる**方式にする（純正の SO-101 は片側だけが動く）
- 固定などの補助を使わず、**指で挟んだ摩擦力だけ**で運ぶ

3. システム構成と手順

Gazebo Harmonic（ヘッドレス・GPU 描画）　SO-101＋2本指グリッパ・剛体ブロック・トレイ・カメラ×2

▲ 関節目標・指の開き幅　　**ros_gz_bridge**　　▼ 画像 160×120・関節状態

ROS 2 ノード（Python 3.12）　collect：IK の手本で操作して記録 ／ eval：観測を推論サーバへ送り、返ってきた行動を指令

データ化・学習
LeRobotDataset → lerobot-train（ACT, GPU）

⇄ **TCP**

推論サーバ
学習済み ACT（Python 3.13＋GPU）

1. **学習データの作成**：逆運動学による手本が「接近 → 下降 → 挟む → 保持 → 持ち上げ → 運搬 → 下降 → 開く → 帰還」を行います。ブロックの位置・向き、置き位置、各区間の速さは毎回ランダムです。成功判定は Gazebo から取得したブロックの実際の位置で行い（トレイ内に置かれ、指が開いていること）、65 回中 60 回の成功分を保存しました。
2. **学習**：LeRobot の ACT を 10,000 ステップ（バッチ 32、約 70 分）学習しました。
3. **評価**：学習に使っていない乱数で 10 か所の配置を作り、自律実行させました。

4. AI モデル：ACT（Action Chunking with Transformers）

ACT（Action Chunking with Transformers）は、人などのお手本の動きを真似して覚える「**模倣学習**」のモデルです。2023 年にスタンフォード大の **ALOHA** という両腕ロボットの研究で発表され、LeRobot に標準で入っています。画像を ResNet-18 で特徴に変換し、Transformer（と CVAE）で「この先の一連の動き」をまとめて予測します。

入力と出力

入力	カメラ画像（今回は上からのカメラ）と、今の 6 つの関節角度です。 補足：カメラは作業台を斜め上から見下ろす位置にあり、画像は 160×120 画素です。6 つの値は、腕の関節角 5 個と指の開き幅 1 個です。ブロックの位置は与えておらず、ポリシーは画像から位置を読み取ります。
出力	この先およそ 1 秒分（数十ステップ）の関節角度の列です。 補足：1 回の推論で 20 ステップ（2 秒分）を予測し、そのうち最初の 10 ステップ（1 秒分）を実行します（下記）。

アクションチャンクの設定（第1版と同じ）

chunk_size = 20	ポリシーは 1 回の推論で、この先 **20 ステップ分**の行動（関節目標 5 つと指の開き幅）をまとめて出力します。
n_action_steps = 10	そのうち**最初の 10 ステップ分だけ**を実際に実行し、残りは捨てて、次の推論で新しく出し直します。
時間に直すと	制御は 10 Hz なので、2 秒先まで予測し、1 秒分を実行して、1 秒ごとに推論し直します（1 回の作業 約 10 秒で約 10 回推論）。

5. 工夫した点

5.1 平行2本指グリッパへの作り替え

- SO-101 の公式モデルから可動ジョーと固定ジョーを取り外し、掌・レールと、左右に平行移動する 2 本の指（直動関節）を URDF に追加しました。2 本の指には同じ指令を送るので、**左右対称に開閉**します。
- 各指は力の上限付きの位置制御（最大 15 N）です。ブロックより内側を目標にすると、指がブロックに当たって止まり、その力で挟みます。
- 指・掌は「ブロック・トレイ・机と衝突する」、腕は「何とも衝突しない」ように、部位ごとに衝突と摩擦を設定しました。

5.2 ラバーパッドとなだらかな指の形状

ハンドの厚みの段階が少なすぎたため、表面にラバーを入れて、もっとなだらかな面にしました。

- **ラバーパッド**：各指の内側（ブロックに触れる面）に、厚さ 3 mm・長さ 36 mm の黒いゴムパッドを付けました（縁は丸め）。摩擦係数は、パッドを 5.0、指の本体（樹脂）を 1.0 にしました。

- **なだらかな形状**：指を根元から先端に向けて、3 段階ではなく **10 段階**で細くしました（根元 10 mm → 先端 6 mm）。先端は丸めています。

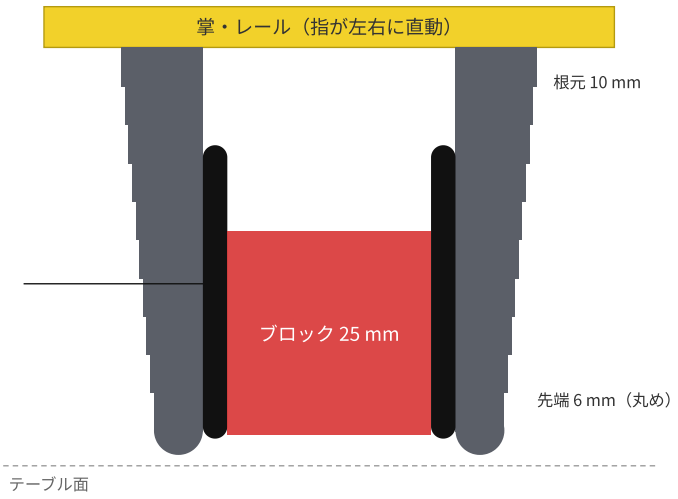


図1　指の断面（模式図）：10 段階のテーパーと内側のラバーパッド

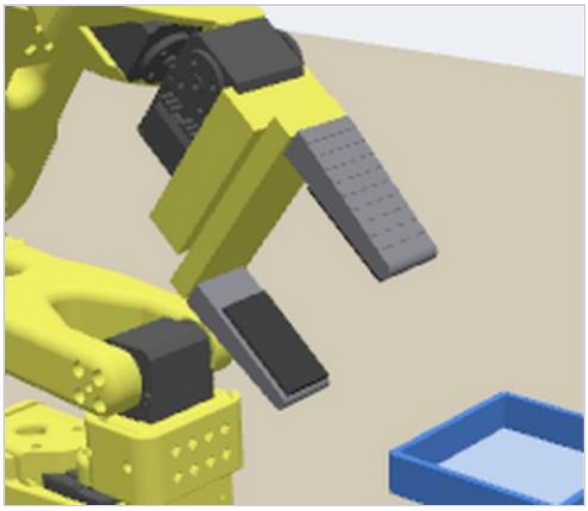


図2　Gazebo 上の指（灰色の本体と黒いラバーパッド）

> **注**：Gazebo の物理エンジンでは、ゴムの柔らかさ（変形）までは再現できないため、ラバーは「摩擦の大きい面」として表現しています。接触計算を安定させるため、衝突判定はパッドと本体の 2 つの箱のままにし、10 段階の形状は見た目に反映しています。

5.3 計測にもとづく把持の調整

ブロックの実際の位置を各段階で記録する診断ツールを作りました。それを使って問題を一つずつ特定し、手本の成功率を上げました。

段階	設定	手本の結果	分かったこと・対策
初期	摩擦 1.5、把持力 8 N	1 / 3	ブロックが指の間で滑り落ちる。トレイの上 8 cm へは手首の可動範囲の都合で届かない（IK 失敗）
調整後	摩擦 5.0、把持力 15 N、動作を遅く、トレイ上 6.5 cm、IK の初期値を複数	4 / 5	運搬中の旋回で滑り出すことがまれにある
ラバーパッド	**パッド 5.0／本体 1.0**	**5 / 5**	**指の間で安定して保持された**

5.4 推論のしかたの比較

推論サーバに、再推論の間隔や時間的アンサンブル（毎ステップ推論し、重なった予測を平均）を切り替えるオプションを付けました。学習し直さずに比較できます（結果は 6.2）。

5.5 評価の観点との対応

創造性	市販アームのハンドを、把持に向いた平行2本指（ラバーパッド・なだらかな指形状）に自分で設計し直した。
技術力	URDF の改造、部位ごとの衝突・摩擦設定、力制御による摩擦把持、診断ツールによる原因の特定、ROS 2 と LeRobot の別 Python 間連携。
実用性	実機のハンドと同じ「挟んで摩擦で持つ」方式。カメラ画像と関節状態だけで動き、物体の位置を事前に知らなくてよい。
表現力	断面図・調整の経過・比較表・コマ送りで、設計の意図と結果を示した。うまくいかなかった点も正直に記録した。

6. 結果

6.1 学習

ステップ	800	1,000	1,200	6,600	8,000	9,200	10,000
損失（L1＋KL）	1.447	1.208	1.018	0.098	0.081	0.070	0.065

6.2 自律実行（同じ 10 か所の配置で比較）

推論のしかた	**成功**	**失敗の内訳**
学習時の設定（10 ステップごとに推論し直す）	4 / 10	つかみ損ね 4、弾き飛ばし 2
5 ステップごとに推論し直す	3 / 10	つかみ損ね 7
時間的アンサンブル（毎ステップ推論し、重なった予測を平均）	**5 / 10**	**つかみ損ね 5（弾き飛ばしなし）**

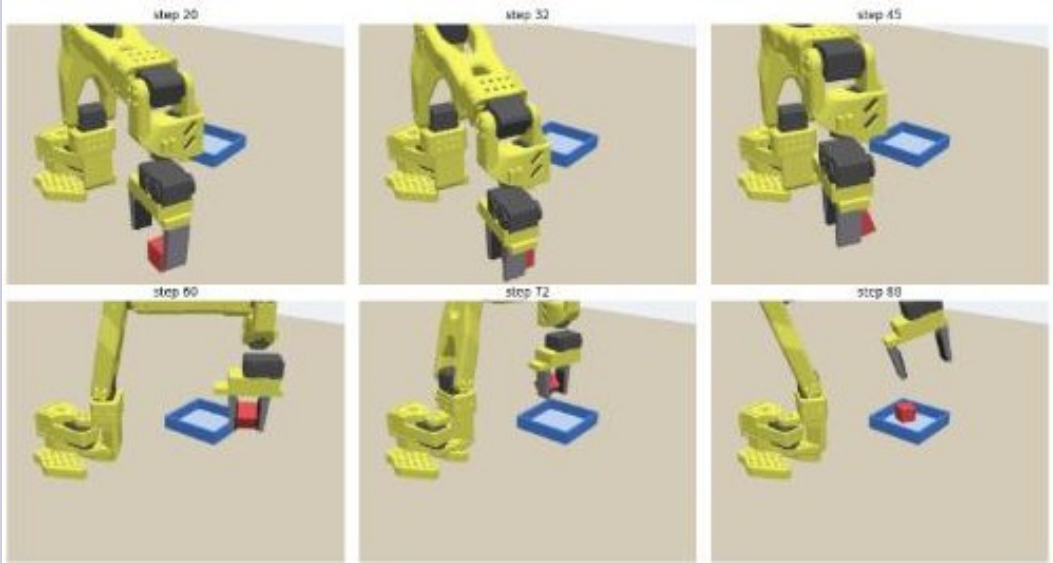


図3　成功例：近づく（20）→ 両指で挟む（32）→ 挟んだまま持ち上げる（45）→ トレイ上へ（60）→ 開いて置く（72）→ 帰還（88）。数字はステップ（0.1 秒単位）

7. 考察と今後

- **摩擦把持は難しい**：成功率は第1版（固定方式、9/10）より下がりました。挟んで持つ方式では、手先の位置や閉じるタイミングのわずかなずれで失敗します。実機で小物をつかむときと同じ難しさが表れました。
- **失敗の主な原因はつかみ損ね**：ブロックが最初の位置に残る失敗が大半でした。手本 60 本では「どこで止まって閉じるか」を覚えるには例が足りないと考えられます。
- **時間的アンサンブル**で動きが滑らかになり、弾き飛ばしがなくなりました。**ラバーパッド**では、手本動作での滑りがなくなりました。指の表面の性質が摩擦把持で重要であることを確認できました。
- **今後**：手本を 120 本程度に増やすこと、つかみ損ねからやり直す動きを手本に入れること、画像の水増しを行うこと。

開発中の反省：Colab のランタイムが切断された際、モデルの保存コマンドが実体ではなくショートカット（シンボリックリンク）だけを保存していたため、モデルと評価映像が失われました。ノートブックは修正済みです。

8. まとめ

SO-101 のハンドを平行2本指グリッパに作り替え、ラバーパッドと 10 段階のなだらかな指の形状にしました。これにより、剛体のブロックを**摩擦力だけで**挟んで運ぶ手本動作を安定させました。これを ACT で模倣学習させ、画像と関節状態だけから「挟む・運ぶ・置く」を自律実行できました（最良の設定で 10 回中 5 回成功）。補助なしの摩擦把持という、より現実に近い難しさに取り組み、課題と改善の方向をはっきりさせました。

参考

- SO-101 モデル：TheRobotStudio/SO-ARM100（Apache-2.0）。グリッパ部分は本課題で改造
- T. Z. Zhao et al., "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware", RSS 2023（ACT・ALOHA）
- LeRobot（Hugging Face）、ROS 2 Jazzy、Gazebo Harmonic、ros_gz